
DSA Hashing Practice

Interview Prep Revision Notes

Hashing & Sliding Window Patterns — Two Sum to Minimum Window Substring

Contents

1. Two Sum
2. Contains Duplicate
3. Valid Anagram
4. Intersection Of Two Arrays
5. Group Anagrams
6. Majority Element
7. Longest Consecutive Sequence
8. Top K Frequent Elements
9. Subarray Sum Equals K
10. Longest Subarray With Sum K
11. 4Sum
13. Subarrays With K Different Integers
14. Count Of Range Sum
15. Minimum Window Substring

Q1 Two Sum

INPUT

```
int[] nums
int target
```

OUTPUT

```
int[] -> two indices
```

```
Example:
nums = [2, 7, 11, 15]
target = 9
```

```
Output = [0, 1]
```

BRUTE FORCE

Har element ko baaki elements ke saath compare karenge.

```
for i = 0 to n
  for j = i + 1 to n
    if nums[i] + nums[j] == target
      return [i, j]
```

Time : $O(n^2)$ Space : $O(1)$

OPTIMIZED APPROACH

HashMap use karenge.

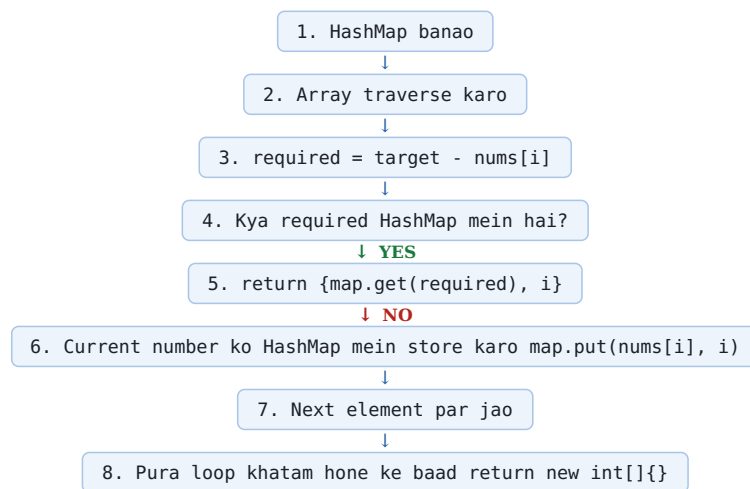
```
HashMap:
number -> index
```

Current element ke liye:

```
required = target - nums[i]
```

Agar required HashMap mein already present hai, to answer mil gaya.

WORKFLOW



CODE

```
import java.util.HashMap;

class Solution {
    public int[] twoSum(int[] nums, int target) {
        HashMap<Integer, Integer> map = new HashMap<>();

        for (int i = 0; i < nums.length; i++) {
            int required = target - nums[i];

            if (map.containsKey(required)) {
                return new int[]{map.get(required), i};
            }

            map.put(nums[i], i);
        }

        return new int[]{};
    }
}
```

COMPLEXITY

Time : $O(n)$ Space : $O(n)$

Q2 Contains Duplicate

INPUT

```
int[] nums
```

OUTPUT

```
boolean -> true / false
```

```
Example:  
nums = [1, 2, 3, 1]
```

```
Output = true
```

BRUTE FORCE

Har element ko baaki elements ke saath compare karenge.

```
for i = 0 to n  
    for j = i + 1 to n  
        if nums[i] == nums[j]  
            return true  
  
return false
```

Time : $O(n^2)$ Space : $O(1)$

OPTIMIZED APPROACH

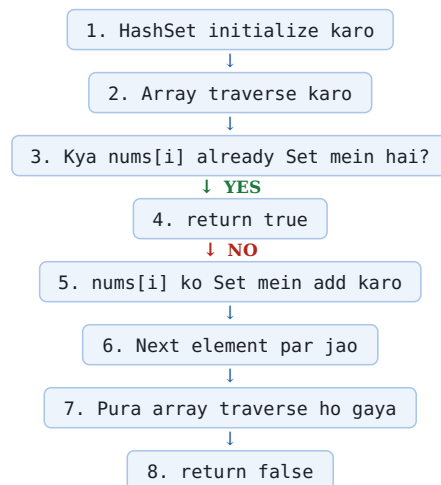
HashSet use karenge.

Humein sirf ye check karna hai:

"Kya ye number pehle aa chuka hai?"

```
HashSet:  
Unique values store karta hai.
```

WORKFLOW



IMPORTANT

```
HashMap -> containsKey()  
HashSet -> contains()
```

CODE

```
import java.util.HashSet;  
  
class Solution {  
    public boolean containsDuplicate(int[] nums) {  
        HashSet<Integer> hs = new HashSet<>();  
        for (int i = 0; i < nums.length; i++) {  
            if (hs.contains(nums[i])) {  
                return true;  
            }  
            hs.add(nums[i]);  
        }  
        return false;  
    }  
}
```

| COMPLEXITY

Time : $O(n)$ Space : $O(n)$

Q3 Valid Anagram

INPUT

```
String s  
String t
```

OUTPUT

```
boolean -> true / false
```

Meaning:

t tabhi s ka anagram hai jab:

1. Dono strings ki length same ho 2. Har character same frequency ke saath present ho

Example:

```
s = "anagram"  
t = "nagaram"
```

```
Output = true
```

Example:

```
s = "rat"  
t = "car"
```

```
Output = false
```

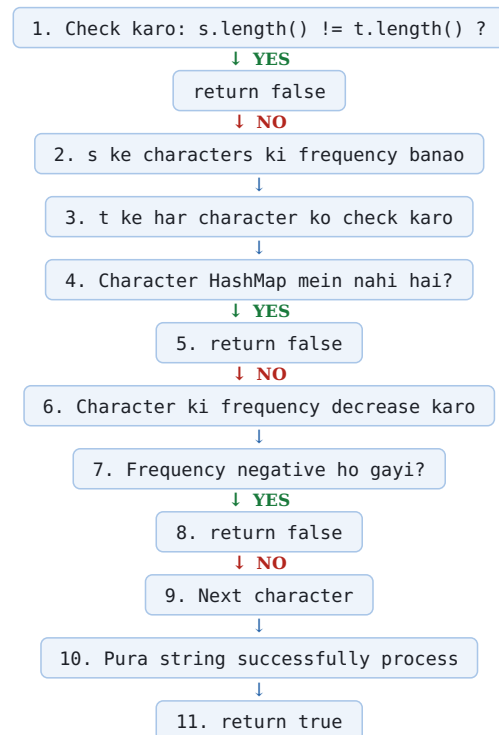
APPROACH

Dono strings ke characters ki frequency count karenge.

HashMap use karenge.

```
HashMap:  
character -> frequency
```

WORKFLOW



CODE

```
import java.util.HashMap;  
  
class Solution {  
  
    public boolean isAnagram(String s, String t) {  
  
        if (s.length() != t.length()) {  
            return false;  
        }  
  
        HashMap<Character, Integer> map = new HashMap<>();  
  
        for (int i = 0; i < s.length(); i++) {  
  
            char ch = s.charAt(i);  
  
            map.put(ch, map.getOrDefault(ch, 0) + 1);  
  

```

```
}

for (int i = 0; i < t.length(); i++) {

    char ch = t.charAt(i);

    if (!map.containsKey(ch)) {
        return false;
    }

    map.put(ch, map.get(ch) - 1);

    if (map.get(ch) < 0) {
        return false;
    }
}

return true;
}
```

COMPLEXITY

Time : $O(n)$ Space : $O(n)$

q4 Intersection Of Two Arrays

INPUT

```
int[] nums1
int[] nums2
```

OUTPUT

```
int[] -> Common elements of both arrays
```

Important: Duplicate elements nahi hone chahiye. Har common element sirf ek baar aayega.

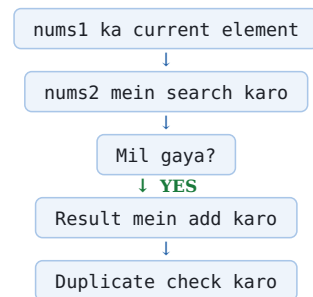
EXAMPLE

```
nums1 = [1, 2, 2, 1]
nums2 = [2, 2]
```

```
Output = [2]
```

BRUTE FORCE

nums1 ke har element ko nums2 mein search karenge.



Time : $O(n \times m)$ Space : $O(1)$ auxiliary

OPTIMIZED APPROACH

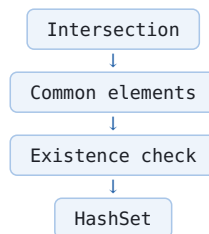
HashSet use karenge.

Humein sirf ye check karna hai:

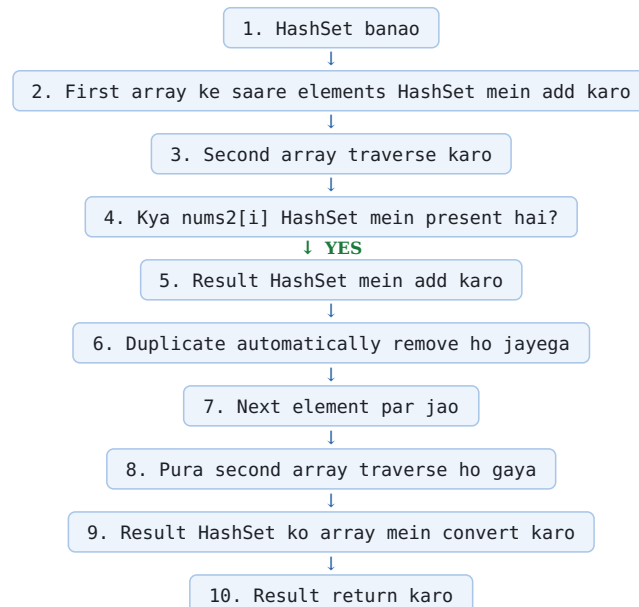
"Kya nums2 ka current element nums1 mein present hai?"

Isliye HashSet use karenge.

PATTERN



WORKFLOW



CODE

```
import java.util.HashSet;
```

```

class Solution {

    public int[] intersection(int[] nums1, int[] nums2) {

        HashSet<Integer> set1 = new HashSet<>();
        HashSet<Integer> resultSet = new HashSet<>();

        // Store nums1 elements
        for (int i = 0; i < nums1.length; i++) {
            set1.add(nums1[i]);
        }

        // Check common elements
        for (int i = 0; i < nums2.length; i++) {

            if (set1.contains(nums2[i])) {
                resultSet.add(nums2[i]);
            }
        }

        // Convert HashSet to array
        Integer[] temp = resultSet.toArray(new Integer[0]);

        int[] result = new int[temp.length];

        for (int i = 0; i < temp.length; i++) {
            result[i] = temp[i];
        }

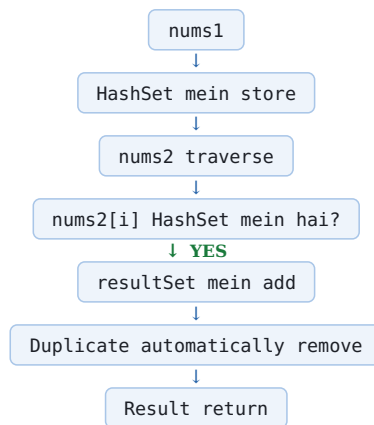
        return result;
    }
}

```

COMPLEXITY

Time : $O(n + m)$ Space : $O(n + m)$

CORE LOGIC



IMPORTANT

HashSet mein duplicate values automatically store nahi hoti.

Example:

```

set.add(2)
set.add(2)
set.add(2)

```

Set mein:

2

sirf ek baar rahega.

Q5 Group Anagrams

INPUT

```
String[] strs
```

OUTPUT

```
List<List<String>>
```

Matlab: Strings ke groups return karne hain. Jo strings anagram hain unko ek hi group mein rakhna hai.

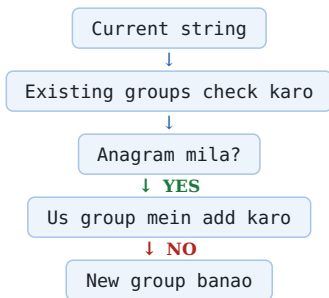
EXAMPLE

Input: ["eat", "tea", "tan", "ate", "nat", "bat"]

Output: [["eat", "tea", "ate"], ["tan", "nat"], ["bat"]]

BRUTE FORCE

Har string ko existing groups ke saath compare karenge.



OPTIMIZED APPROACH

HashMap use karenge.

HashMap:

KEY → Sorted string

VALUE → Us key ke anagram strings ki List

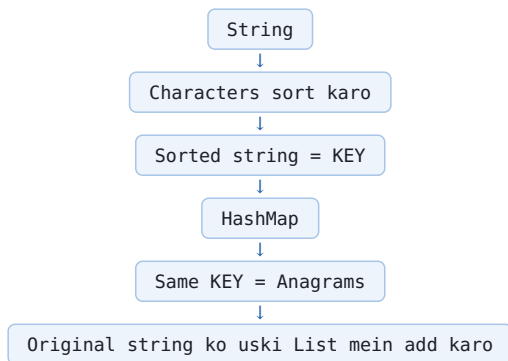
Example:

```
eat → aet
tea → aet
ate → aet
```

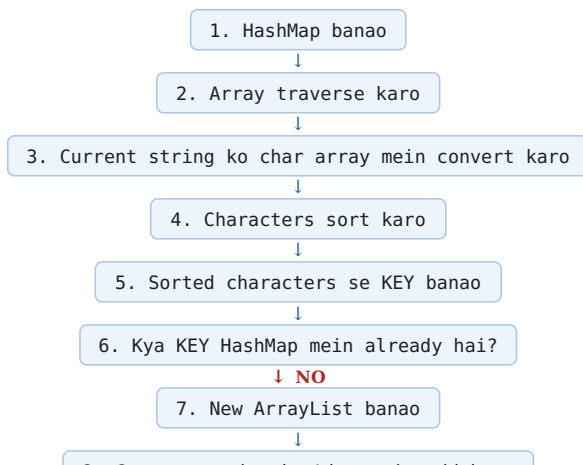
So:

```
aet → ["eat", "tea", "ate"]
```

PATTERN



WORKFLOW



8. Current string ko List mein add karo



9. Next string



10. Pura array traverse hone ke baad HashMap ki values return karo

CODE

```
import java.util.*;

class Solution {

    public List<List<String>> groupAnagrams(String[] strs) {

        HashMap<String, List<String>> map = new HashMap<>();

        for (int i = 0; i < strs.length; i++) {

            char[] chars = strs[i].toCharArray();

            Arrays.sort(chars);

            String key = new String(chars);

            if (!map.containsKey(key)) {
                map.put(key, new ArrayList<>());
            }

            map.get(key).add(strs[i]);

        }

        return new ArrayList<>(map.values());

    }

}
```

COMPLEXITY

Time : $O(n \times k \log k)$ Space : $O(n \times k)$

n = number of strings
k = average length of each string

IMPORTANT

List<String>
= Strings ki ek list

Example: ["eat", "tea", "ate"]

List<List<String>>
= Lists ki list

Example: [["eat", "tea", "ate"], ["tan", "nat"], ["bat"]]

HashMap<String, List<String>>

KEY

Sorted string

VALUE

Anagram strings ki list

Q6 Majority Element

INPUT

```
int[] nums
```

OUTPUT

```
int
```

Matlab: Jo element array mein $n/2$ se zyada baar aata hai, us element ko return karna hai.

EXAMPLE

```
nums = [3, 2, 3]
```

```
n = 3  
n/2 = 1
```

```
3 → 2 times
```

$2 > 1$

Output: 3

ANOTHER EXAMPLE

```
nums = [2, 2, 1, 1, 1, 2, 2]
```

```
n = 7  
n/2 = 3
```

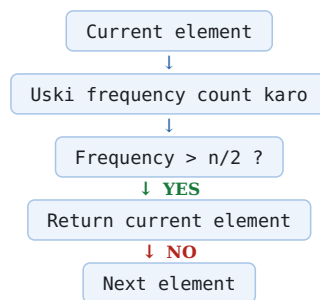
```
2 → 4 times
```

$4 > 3$

Output: 2

BRUTE FORCE

Har element ki frequency count karo.



Time : $O(n^2)$ Space : $O(1)$

OPTIMIZED APPROACH

```
HashMap use karenge.
```

```
HashMap:
```

```
number → frequency
```

Example:

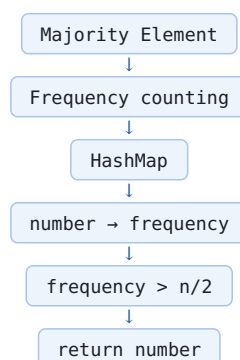
```
nums = [2, 2, 1, 1, 1]
```

```
Map:
```

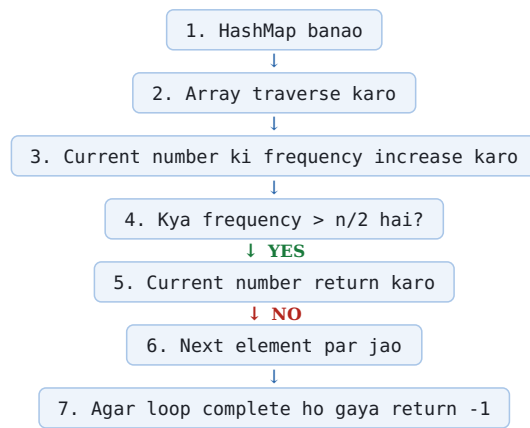
```
2 → 2  
1 → 3
```

Jaise hi kisi number ki frequency $n/2$ se greater ho jaaye, usi time return kar denge.

PATTERN



WORKFLOW



IMPORTANT CODE

```
map.put(
    nums[i],
    map.getOrDefault(nums[i], 0) + 1
);
```

Iska matlab:

Current number ki frequency
1 se increase karo.

MAJORITY CHECK

```
if (map.get(nums[i]) > nums.length / 2) {
    return nums[i];
}
```

COMPLETE CODE

```
import java.util.HashMap;

class Solution {

    public int majorityElement(int[] nums) {

        HashMap<Integer, Integer> map = new HashMap<>();

        for (int i = 0; i < nums.length; i++) {

            map.put(nums[i], map.getOrDefault(nums[i], 0) + 1);

            if (map.get(nums[i]) > nums.length / 2) {
                return nums[i];
            }
        }

        return -1;
    }
}
```

COMPLEXITY

Time : O(n) Space : O(n)

Q7 Longest Consecutive Sequence

INPUT

```
int[] nums
```

OUTPUT

```
int
```

Matlab:
Longest consecutive sequence ki length return karni hai.

EXAMPLE

```
nums = [100, 4, 200, 1, 3, 2]
```

Consecutive sequence:

```
1 -> 2 -> 3 -> 4
```

```
Answer = 4
```

ANOTHER EXAMPLE

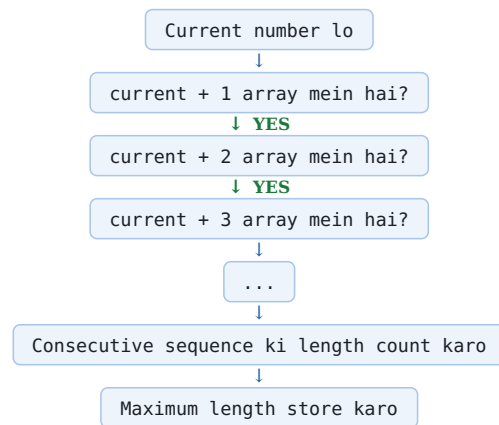
```
nums = [0, 3, 7, 2, 5, 8, 4, 6, 0, 1]
```

Sequence:

```
0 -> 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7 -> 8
```

```
Answer = 9
```

BRUTE FORCE



Time : $O(n^2)$ Space : $O(1)$

OPTIMIZED APPROACH

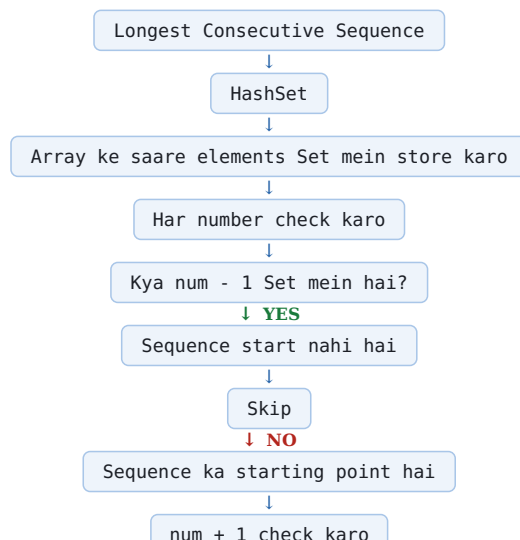
HashSet use karenge .

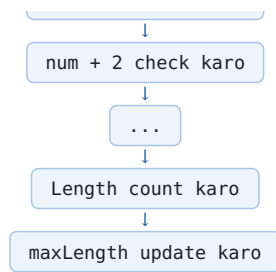
Reason:

Humein quickly check karna hai ki koi number Set mein present hai ya nahi.

HashSet average lookup:
 $O(1)$

PATTERN





IMPORTANT CONDITION

```
if (!set.contains(num - 1))
```

Meaning:

Agar num - 1 Set mein nahi hai, to current num sequence ka STARTING POINT hai.

EXAMPLE

```
nums = [100, 4, 200, 1, 3, 2]
```

```
For num = 1:
```

```
num - 1 = 0
```

0 Set mein nahi hai.

Therefore:

1 sequence ka starting point hai.

Then:

```
1 -> 2 -> 3 -> 4
```

```
Length = 4
```

```
For num = 2:
```

```
num - 1 = 1
```

1 Set mein already hai.

Therefore:

2 se sequence start nahi karenge.

WORKFLOW



CODE

```
import java.util.HashSet;

class Solution {
```

```
public int longestConsecutive(int[] nums) {  
  
    HashSet<Integer> set = new HashSet<>();  
  
    for (int i = 0; i < nums.length; i++) {  
        set.add(nums[i]);  
    }  
  
    int maxLength = 0;  
  
    for (int i = 0; i < nums.length; i++) {  
  
        int num = nums[i];  
  
        if (!set.contains(num - 1)) {  
  
            int current = num;  
            int length = 1;  
  
            while (set.contains(current + 1)) {  
                current++;  
                length++;  
            }  
  
            maxLength = Math.max(maxLength, length);  
        }  
    }  
  
    return maxLength;  
}
```

COMPLEXITY

Time : $O(n)$ average Space : $O(n)$

QUICK NOTE

Problem: Longest Consecutive Sequence

Pattern:
HashSet

Core Logic: num - 1 nahi mila



Sequence start

Then: num + 1, num + 2, num + 3... check karo

Complexity: $O(n)$ average time $O(n)$ space

IMPORTANT

HashSet:
Fast existence checking

contains(): Element present hai ya nahi check karta hai.

Example:

```
set.contains(5)  
  
true  -> 5 present hai  
false -> 5 present nahi hai
```

Q8 Top K Frequent Elements

INPUT

```
int[] nums
int k
```

OUTPUT

```
int[]
```

Matlab: Array mein jo k numbers sabse zyada baar aaye hain, unko return karna hai.

EXAMPLE

```
nums = [1, 1, 1, 2, 2, 3]
k = 2
```

Frequency:

```
1 -> 3
2 -> 2
3 -> 1
```

Top 2 frequent elements:

[1, 2]

BRUTE FORCE

1. Har number ki frequency count karo 2. Frequency ke according numbers ko sort karo 3. Top k numbers return karo

OPTIMIZED APPROACH

Abhi Heap/PriorityQueue use nahi karenge.

HashMap use karenge:

number -> frequency

WORKFLOW



FREQUENCY CODE

```
map.put(
    nums[i],
    map.getOrDefault(nums[i], 0) + 1
);
```

MEANING

Current number ki frequency 1 se increase karo.

COMPLETE CODE

```
import java.util.*;

class Solution {

    public int[] topKFrequent(int[] nums, int k) {

        HashMap<Integer, Integer> map = new HashMap<>();

        // Frequency count
        for (int i = 0; i < nums.length; i++) {
            map.put(nums[i], map.getOrDefault(nums[i], 0) + 1);
        }

        // Unique elements ka array
        int[] elements = new int[map.size()];
```



```

int index = 0;

for (int num : map.keySet()) {
    elements[index++] = num;
}

// Frequency ke according sort
for (int i = 0; i < elements.length; i++) {

    for (int j = i + 1; j < elements.length; j++) {

        if (map.get(elements[j]) > map.get(elements[i])) {

            int temp = elements[i];

            elements[i] = elements[j];

            elements[j] = temp;

        }
    }
}

// Top k elements
int[] result = new int[k];

for (int i = 0; i < k; i++) {
    result[i] = elements[i];
}

return result;
}
}

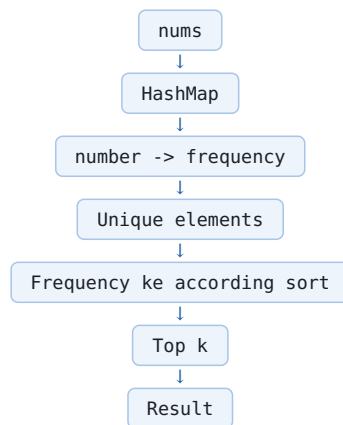
```

COMPLEXITY

Let m = number of unique elements.

Time : $O(n + m^2)$ Space : $O(m + k)$

CORE LOGIC



IMPORTANT

HashMap:
number -> frequency

HashSet:
sirf existence check

Top K Frequent: frequency count karne ke liye HashMap use hota hai.

QUICK NOTE

Problem: Top K Frequent Elements

Pattern:
HashMap + Frequency + Sorting

Core:
number -> frequency

Then: frequency ke according sort

Then: first k elements return

Time: $O(n + m^2)$

Space: $O(m + k)$

Q9 Subarray Sum Equals K

INPUT

```
int[] nums
int k
```

OUTPUT

```
int
```

Matlab: Kitne continuous subarrays ka sum exactly k hai, unka count return karna hai.

EXAMPLE

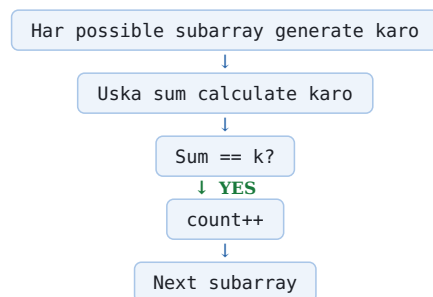
```
nums = [1, 1, 1]
k = 2
```

Subarrays:

```
[1, 1] -> 2
[1, 1] -> 2
```

```
Answer = 2
```

BRUTE FORCE



Example:

```
nums = [1, 2, 3]
k = 3
```

Possible subarrays:

```
[1]      -> 1
[1,2]    -> 3  YES
[1,2,3]  -> 6
[2]      -> 2
[2,3]    -> 5
[3]      -> 3  YES
```

```
Answer = 2
```

BRUTE FORCE COMPLEXITY

Time : $O(n^2)$ Space : $O(1)$

OPTIMIZED APPROACH

Prefix Sum + HashMap

PREFIX SUM

Prefix Sum ka matlab:

Array ko left se traverse karte hue current position tak ka total sum.

Example:

```
nums = [1, 2, 3, 4]
```

Prefix sums:

```
1
1 + 2 = 3
1 + 2 + 3 = 6
1 + 2 + 3 + 4 = 10
```

So:

[1, 3, 6, 10]

IMPORTANT FORMULA

```
currentPrefixSum - k = previousPrefixSum
```

Agar previousPrefixSum HashMap mein hai, to ek ya multiple subarrays ka sum k hai.

EXAMPLE

```
nums = [1, 1, 1]
k = 2
```

Start:

```
prefixSum = 0

HashMap:

0 -> 1

i = 0

nums[i] = 1

prefixSum = 1

required = prefixSum - k
          = 1 - 2
          = -1
```

-1 map mein nahi hai.

Store:

```
1 -> 1

i = 1

nums[i] = 1

prefixSum = 2

required = 2 - 2
          = 0
```

0 map mein hai.

```
0 ki frequency = 1

count = 1

i = 2

nums[i] = 1

prefixSum = 3

required = 3 - 2
          = 1
```

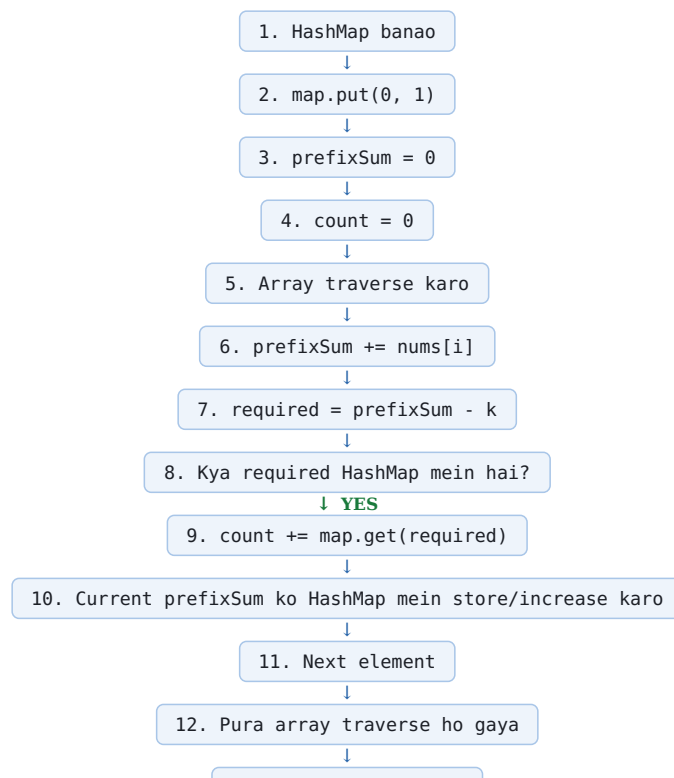
1 map mein hai.

```
1 ki frequency = 1

count = 2

Answer = 2
```

WORKFLOW



IMPORTANT CODE

```
map.put(0, 1);
```

Meaning:

Starting mein prefix sum 0
ek baar available hai.

Ye important hai taaki
array ke starting se banne wale
subarray ko bhi count kar saken.

FREQUENCY UPDATE

```
map.put(
    prefixSum,
    map.getOrDefault(prefixSum, 0) + 1
);
```

COMPLETE CODE

```
import java.util.HashMap;

class Solution {

    public int subarraySum(int[] nums, int k) {

        HashMap<Integer, Integer> map = new HashMap<>();

        map.put(0, 1);

        int prefixSum = 0;
        int count = 0;

        for (int i = 0; i < nums.length; i++) {

            prefixSum += nums[i];

            int required = prefixSum - k;

            if (map.containsKey(required)) {
                count += map.get(required);
            }

            map.put(
                prefixSum,
                map.getOrDefault(prefixSum, 0) + 1
            );
        }

        return count;
    }
}
```

COMPLEXITY

Time : $O(n)$ Space : $O(n)$

QUICK NOTE

Problem: Subarray Sum Equals K

Output: Count of subarrays

Pattern: Prefix Sum + HashMap

Core:

```
prefixSum += nums[i]

required = prefixSum - k

if map contains required:
    count += map.get(required)

Important:
map.put(0, 1)
```

Complexity: $O(n)$ time $O(n)$ space

Q10 Longest Subarray With Sum K

INPUT

```
int[] nums
int k
```

OUTPUT

```
int
```

Matlab: Aise continuous subarray ki maximum length return karni hai jiska sum exactly k ho.

EXAMPLE

```
nums = [1, 2, 3, 1, 1, 1]
k = 6
```

Possible subarray:

[1, 2, 3]

```
Sum = 6
```

```
Length = 3
```

Another:

```
nums = [1, 2, 1, 1, 1, 1]
k = 3
```

Possible subarrays:

```
[1, 2] -> sum 3 -> length 2
```

```
[1, 1, 1] -> sum 3 -> length 3
```

```
Answer = 3
```

BRUTE FORCE

Har possible subarray ka sum calculate karo



Sum == k?

↓ **YES**

Uski length calculate karo



Maximum length update karo

Example:

```
nums = [1, 2, 3]
k = 3

[1]      -> 1
[1,2]    -> 3 -> length 2
[1,2,3]  -> 6
[2]      -> 2
[2,3]    -> 5
[3]      -> 3 -> length 1

Maximum length = 2
```

BRUTE FORCE COMPLEXITY

Time : $O(n^2)$ Space : $O(1)$

OPTIMIZED APPROACH

Prefix Sum + HashMap

IMPORTANT DIFFERENCE

Q9

Subarray Sum Equals K

Humein COUNT chahiye.

Q10

Longest Subarray With Sum K

Humein MAXIMUM LENGTH chahiye.

PREFIX SUM

Example:

```
nums = [1, 2, 3]
```

Prefix:

IMPORTANT FORMULA

```
currentPrefixSum - k = previousPrefixSum
```

Agar required prefix sum HashMap mein milta hai, to current index aur previous index ke beech subarray ka sum k hai.

IMPORTANT DIFFERENCE FROM Q9

Q9 mein:
prefixSum -> frequency store karte hain.

Q10 mein:
prefixSum -> FIRST INDEX store karte hain.

Kyun?

Humein longest subarray chahiye.

Agar same prefix sum dobara aaye, to sabse pehla index use karna hai taaki length maximum ho.

EXAMPLE

```
nums = [1, 2, 1, 1, 1]
k = 3
```

Start:

```
map = {}

prefixSum = 0

i = 0

prefixSum = 1

required = 1 - 3
          = -2
```

-2 nahi hai.

Store:

```
1 -> 0

i = 1

prefixSum = 3

required = 3 - 3
          = 0
```

0 map mein hai.

Starting index -1 maana jayega.

Length:

```
i - (-1)
= 1 - (-1)
= 2

maxLength = 2

i = 2

prefixSum = 4

required = 4 - 3
          = 1
```

1 map mein hai.

```
Index = 0
```

Length:

```
2 - 0
= 2

i = 3

prefixSum = 5

required = 5 - 3
          = 2
```

2 map mein nahi hai.

```
i = 4

prefixSum = 6
```

```
required = 6 - 3
          = 3
```

3 map mein index 1 par hai.

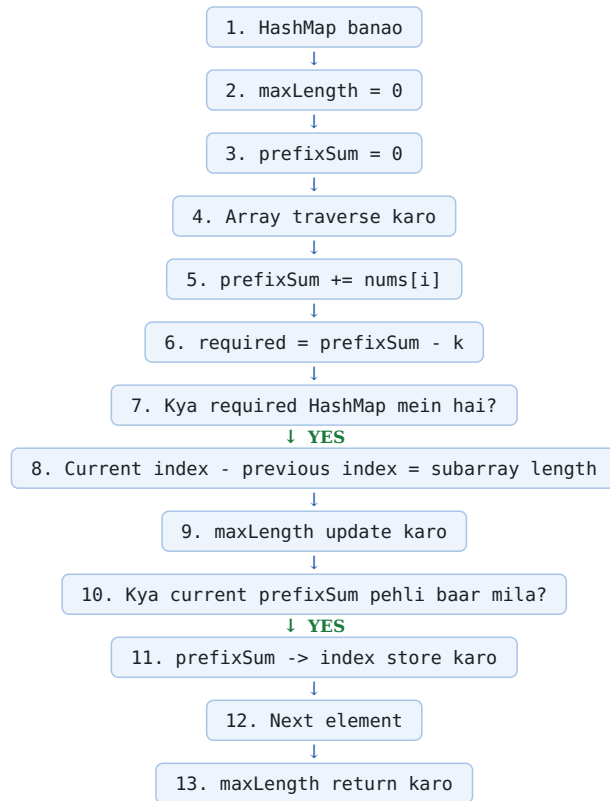
Length:

```
4 - 1
= 3

maxLength = 3

Answer = 3
```

WORKFLOW



IMPORTANT

```
HashMap mein:

prefixSum -> FIRST INDEX
```

Isliye:

```
if (!map.containsKey(prefixSum)) {
    map.put(prefixSum, i);
}
```

Why only first index?

Example:

```
prefixSum = 5

Pehli baar:
index = 2

Baad mein:
index = 6
```

Longest subarray ke liye index 2 better hai, kyunki distance zyada milega.

COMPLETE CODE

```
import java.util.HashMap;

class Solution {

    public int longestSubarray(int[] nums, int k) {

        HashMap<Integer, Integer> map = new HashMap<>();

        int prefixSum = 0;
        int maxLength = 0;

        for (int i = 0; i < nums.length; i++) {

            prefixSum += nums[i];
```

```

        if (prefixSum == k) {
            maxLength = i + 1;
        }

        int required = prefixSum - k;

        if (map.containsKey(required)) {

            int length = i - map.get(required);

            maxLength = Math.max(maxLength, length);
        }

        if (!map.containsKey(prefixSum)) {
            map.put(prefixSum, i);
        }
    }

    return maxLength;
}
}

```

COMPLEXITY

Time : O(n) Space : O(n)

QUICK NOTE

Problem: Longest Subarray With Sum K

Output: Maximum length

Pattern: Prefix Sum + HashMap

HashMap:
prefixSum -> first index

Core:

```

prefixSum += nums[i]

required = prefixSum - k

```

if map contains required:

```

length = i - map.get(required)

maxLength = Math.max(maxLength, length)

```

IMPORTANT DIFFERENCE

SUBARRAY SUM EQUALS K

Goal: COUNT

HashMap:
prefixSum -> frequency

Formula:
count += map.get(prefixSum - k)

LONGEST SUBARRAY WITH SUM K

Goal: MAXIMUM LENGTH

HashMap:
prefixSum -> first index

Formula:
length = i - map.get(prefixSum - k)

Q11 4Sum

INPUT

```
int[] nums
int target
```

OUTPUT

```
List<List<Integer>>
```

Matlab: 4 aise numbers ka group return karna hai jinka sum target ke equal ho.

Har group mein exactly 4 elements honge.

Duplicate quadruplets nahi hone chahiye.

EXAMPLE

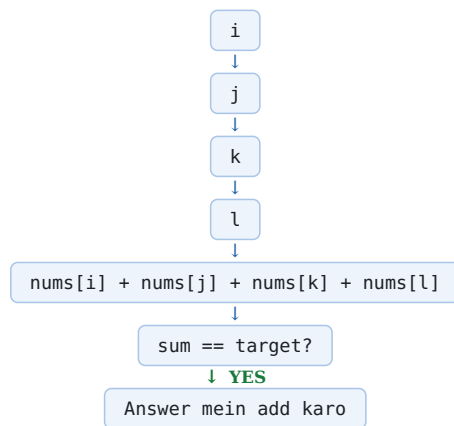
```
nums = [1, 0, -1, 0, -2, 2]
target = 0
```

Output:

```
[ [-2, -1, 1, 2], [-2, 0, 0, 2], [-1, 0, 0, 1] ]
```

BRUTE FORCE

4 loops use karenge.



BRUTE FORCE COMPLEXITY

Time : $O(n^4)$ Space : $O(1)$ extra

OPTIMIZED APPROACH

Sorting + Two Pointers

MAIN IDEA

4 numbers chahiye:

```
nums[i] + nums[j] + nums[p] + nums[q] = target
```

Pehle 2 numbers fix karenge:

$\text{nums}[i]$ $\text{nums}[j]$

Ab remaining 2 numbers find karenge:

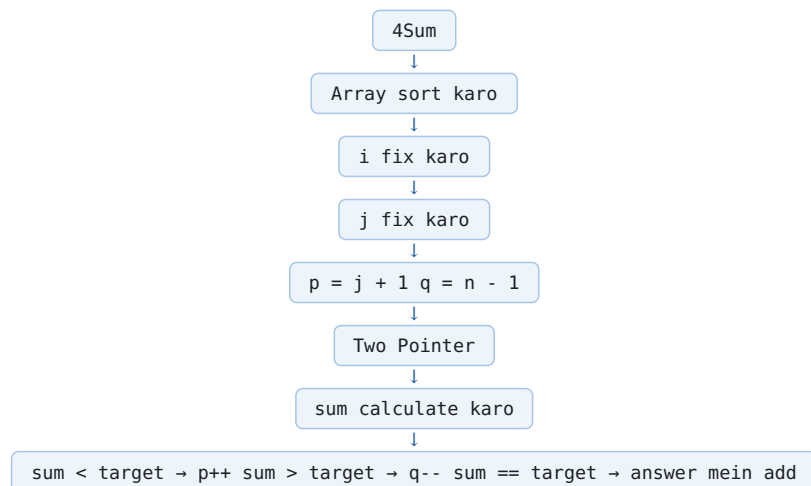
$\text{nums}[p] + \text{nums}[q]$

Formula:

$\text{nums}[p] + \text{nums}[q] = \text{target} - \text{nums}[i] - \text{nums}[j]$

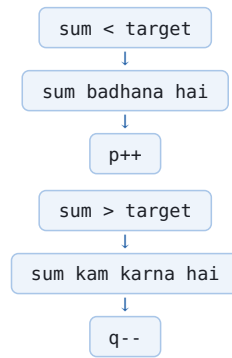
Isliye remaining 2 numbers ko Two Pointer se find kar sakte hain.

PATTERN

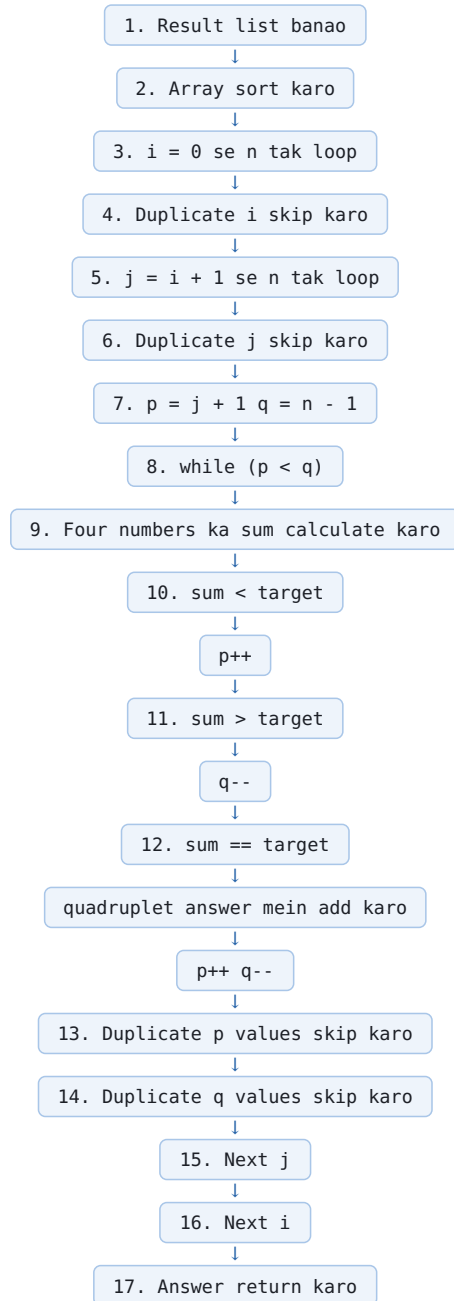


WHY SORTING?

Array sorted hone ki wajah se:



WORKFLOW



DUPLICATE HANDLING

i ke liye:

```
if (i > 0 && nums[i] == nums[i - 1]) {
    continue;
}
```

j ke liye:

```
if (j > i + 1 && nums[j] == nums[j - 1]) {
    continue;
}
```

p ke liye:

```
while (p < q && nums[p] == nums[p - 1]) {
    p++;
}
```

q ke liye:

```
while (p < q && nums[q] == nums[q + 1]) {
    q--;
}
```

COMPLETE CODE

```
import java.util.*;

class Solution {

    public List<List<Integer>> fourSum(int[] nums, int target) {

        List<List<Integer>> ans = new ArrayList<>();

        int n = nums.length;

        Arrays.sort(nums);

        for (int i = 0; i < n; i++) {

            if (i > 0 && nums[i] == nums[i - 1]) {
                continue;
            }

            for (int j = i + 1; j < n; j++) {

                if (j > i + 1 && nums[j] == nums[j - 1]) {
                    continue;
                }

                int p = j + 1;
                int q = n - 1;

                while (p < q) {

                    long sum = (long) nums[i]
                        + nums[j]
                        + nums[p]
                        + nums[q];

                    if (sum < target) {
                        p++;
                    }

                    else if (sum > target) {
                        q--;
                    }

                    else {

                        ans.add(Arrays.asList(
                            nums[i],
                            nums[j],
                            nums[p],
                            nums[q]
                        ));

                        p++;
                        q--;

                        while (p < q && nums[p] == nums[p - 1]) {
                            p++;
                        }

                        while (p < q && nums[q] == nums[q + 1]) {
                            q--;
                        }

                    }

                }

            }

        }

        return ans;
    }
}
```

COMPLEXITY

Time: $O(n^3)$

Reason:

```
i loop      → O(n)
j loop      → O(n)
Two Pointer → O(n)
```

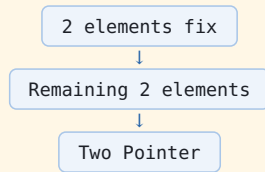
Total: $O(n \times n \times n) = O(n^3)$

Space: $O(1)$ extra space

Output list ko auxiliary space mein count nahi karte.

IMPORTANT

4Sum ka core:

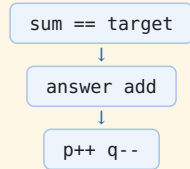


$i + j + p + q$

Two Pointer:

```
sum < target → p++
```

```
sum > target → q--
```



QUICK REVISION

Problem: 4Sum

Pattern: Sorting + Two Pointers

Brute Force: 4 loops

Brute Force Time: $O(n^4)$

Optimized: 2 loops + Two Pointer

Optimized Time: $O(n^3)$

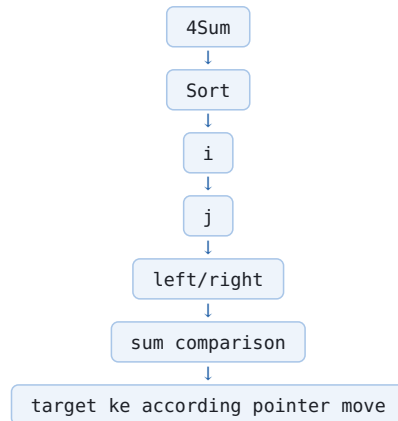
Extra Space: $O(1)$

CONNECTION WITH TWO SUM

Two Sum: 2 numbers find karo

4Sum: 2 numbers fix karo + remaining 2 numbers Two Pointer se find karo

GOLDEN PATTERN



Q13 Subarrays With K Different Integers

INPUT

```
int[] nums
int k
```

OUTPUT

```
int
```

Matlab: Kitne continuous subarrays mein exactly k different/distinct integers hain, unka count return karna hai.

EXAMPLE

```
nums = [1, 2, 1, 2, 3]
k = 2
```

Valid subarrays:

[1,2] [2,1] [1,2]

[1,2,1] [2,1,2]

[1,2,1,2] [2,1,2]

Total:

```
Answer = 7
```

IMPORTANT

Different integers ka matlab unique numbers.

Example:

[1,2,1,2]

Different numbers:

1, 2

```
Total different = 2
```

[1,1,1]

Different numbers:

1

```
Total different = 1
```

SUBARRAY

Subarray continuous hona chahiye.

Example:

[1,2,3]

Valid:

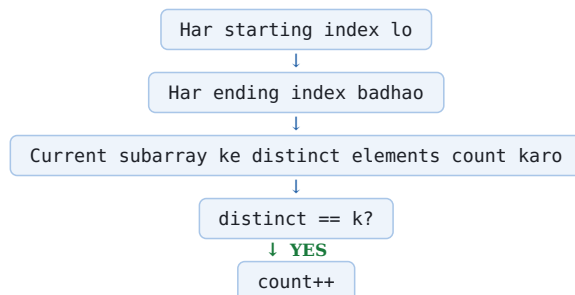
[1] [2] [3]

[1,2] [2,3]

[1,2,3]

[1,3] valid nahi hai kyunki 2 ko skip kiya gaya hai.

BRUTE FORCE



BRUTE FORCE COMPLEXITY

Time : $O(n^2)$ Space : $O(k)$ / $O(n)$

OPTIMIZED APPROACH

```
HashMap + Sliding Window
```

IMPORTANT TRICK

Exactly K = At Most K - At Most K - 1

Formula:

$\text{exactlyK}(\text{nums}, k) = \text{atMost}(\text{nums}, k) - \text{atMost}(\text{nums}, k - 1)$

WHY?

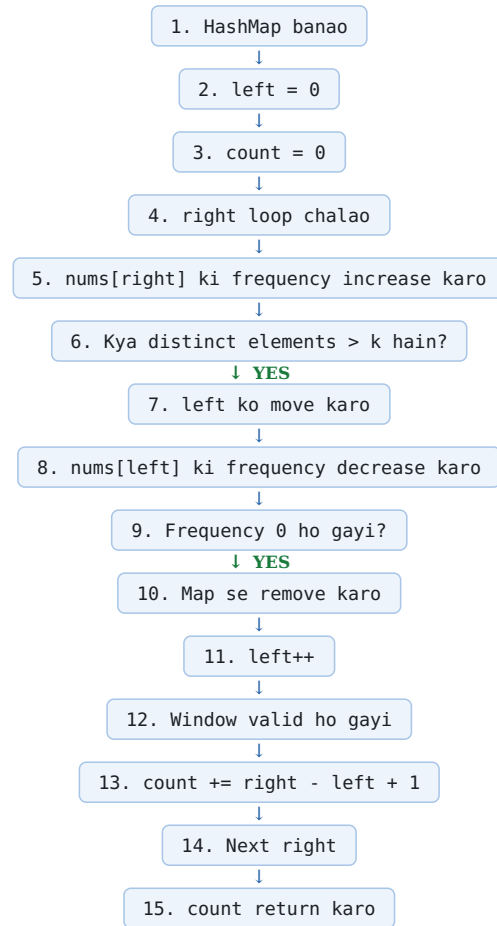
atMost(k) mein:
1 distinct 2 distinct ... k distinct
sab included hain.
atMost(k - 1) mein:
1 distinct 2 distinct ... k - 1 distinct
included hain.
Subtract karne ke baad:
atMost(k) - atMost(k - 1)
sirf exactly k distinct wale subarrays bachenge.

AT MOST K

Goal:

Maximum K different integers wale subarrays count karna.

WORKFLOW



WHY

```
count += right - left + 1
```

Example:

```
left = 2  
right = 5
```

Current window:

[2 5]

Valid subarrays ending at right:

[left ... right] [left+1 ... right] [left+2 ... right]

Total:

right - left + 1

```
= 5 - 2 + 1
```

```
= 4
```

COMPLETE CODE

```
import java.util.HashMap;  
  
class Solution {  
  
    public int subarraysWithKDistinct(int[] nums, int k) {  
  
        return atMost(nums, k) - atMost(nums, k - 1);  
    }  
}
```

```

    }

    public int atMost(int[] nums, int k) {

        HashMap<Integer, Integer> map = new HashMap<>();

        int left = 0;
        int count = 0;

        for (int right = 0; right < nums.length; right++) {

            map.put(
                nums[right],
                map.getOrDefault(nums[right], 0) + 1
            );

            while (map.size() > k) {

                map.put(
                    nums[left],
                    map.get(nums[left]) - 1
                );

                if (map.get(nums[left]) == 0) {
                    map.remove(nums[left]);
                }

                left++;
            }

            count += right - left + 1;
        }

        return count;
    }
}

```

COMPLEXITY

Time: O(n)

Space: O(k) approximately

QUICK NOTE

Problem: Subarrays with K Different Integers

Pattern:
HashMap + Sliding Window

Core Formula:

Exactly K = At Most K - At Most K - 1

Core Function:

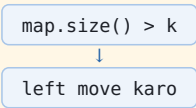
atMost(nums, k)

HashMap:
number -> frequency

Sliding Window:

left right

Important:



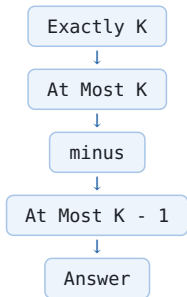
Frequency 0:

map.remove(nums[left])

Most Important:

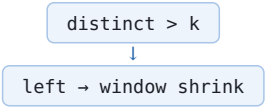
count += right - left + 1

FINAL REVISION



Sliding Window:

```
right →  
window expand
```



Q14 Count Of Range Sum

INPUT

```
int[] nums
int lower
int upper
```

OUTPUT

```
int / long
```

Matlab: Kitne continuous subarrays ka sum lower aur upper ke beech hai, unka count return karna hai.

Condition:

```
lower <= subarray sum <= upper
```

EXAMPLE

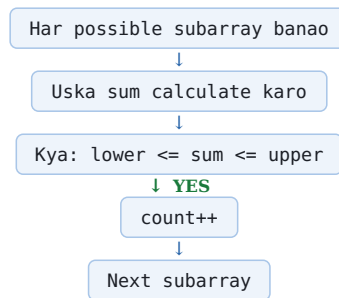
```
nums = [-2, 5, -1]
lower = -2
upper = 2
```

Subarrays:

```
[-2]      -> -2   -> VALID
[5]       -> 5    -> INVALID
[-1]      -> -1   -> VALID
[-2, 5]   -> 3    -> INVALID
[5, -1]   -> 4    -> INVALID
[-2, 5, -1] -> 2  -> VALID
```

```
Answer = 3
```

BRUTE FORCE



BRUTE FORCE

2 loops use kar sakte hain:

```
i = starting point
j = ending point
```

sum maintain karenge.

Time: $O(n^2)$

Space: $O(1)$

OPTIMIZED APPROACH

Prefix Sum + Merge Sort

WHY PREFIX SUM?

Subarray sum ko prefix sum se quickly represent kar sakte hain.

Example:

```
nums = [-2, 5, -1]
```

Prefix Sum:

```
0 -2 3 2
```

Agar:

```
prefix[i] = current prefix
prefix[j] = previous prefix
```

Subarray sum:

```
prefix[j] - prefix[i]
```

Isliye humein check karna hai:

```
lower <= prefix[j] - prefix[i] <= upper
```

IMPORTANT FORMULA

```
lower <= currentPrefix - previousPrefix <= upper
```

Rearrange:

```
currentPrefix - upper
<= previousPrefix
<=
currentPrefix - lower
```

Example:

```
currentPrefix = 2
lower = -2
upper = 2

2 - 2 <= previousPrefix <= 2 - (-2)

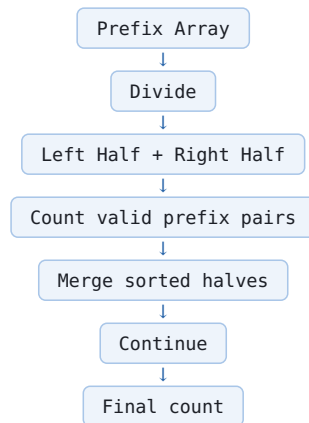
0 <= previousPrefix <= 4
```

Agar previous prefix sum 0 se 4 ke beech hai, to current subarray ka sum valid hai.

WHY MERGE SORT?

Prefix sums ko sort karne ke baad merge sort ke during efficiently count kar sakte hain ki kitne previous prefix sums given range mein hain.

MAIN IDEA



PREFIX SUM ARRAY

Original:

[-2, 5, -1]

Prefix:

[0, -2, 3, 2]

Har valid subarray:

currentPrefix - previousPrefix

se represent hota hai.

WORKFLOW



| TWO POINTER IDEA

Merge step mein left half aur right half already sorted hain.

For every right prefix:

lower bound:

rightPrefix - upper

upper bound:

rightPrefix - lower

Left half mein count karo:

```
lower bound <= leftPrefix <= upper bound
```

Do pointers use kar sakte hain:

low high

low: range ke andar aane wala first index

high: range se bahar hone wala first index

Valid elements:

high - low

| COMPLETE CODE

```
class Solution {

    public int countRangeSum(int[] nums, int lower, int upper) {

        long[] prefix = new long[nums.length + 1];

        for (int i = 0; i < nums.length; i++) {
            prefix[i + 1] = prefix[i] + nums[i];
        }

        return (int) mergeSort(prefix, 0, prefix.length, lower, upper);
    }

    public long mergeSort(
        long[] prefix,
        int left,
        int right,
        int lower,
        int upper) {

        if (right - left <= 1) {
            return 0;
        }

        int mid = left + (right - left) / 2;

        long count = 0;

        count += mergeSort(
            prefix,
            left,
            mid,
            lower,
            upper
        );

        count += mergeSort(
            prefix,
            mid,
            right,
            lower,
            upper
        );

        int low = mid;
        int high = mid;

        for (int i = left; i < mid; i++) {

            while (
                low < right &&
                prefix[low] - prefix[i] < lower
            ) {
                low++;
            }

            while (
                high < right &&
                prefix[high] - prefix[i] <= upper
            ) {
                high++;
            }

            count += high - low;
        }
    }
}
```

```

        long[] temp = new long[right - left];

        int i = left;
        int j = mid;
        int k = 0;

        while (i < mid && j < right) {

            if (prefix[i] <= prefix[j]) {
                temp[k++] = prefix[i++];
            } else {
                temp[k++] = prefix[j++];
            }
        }

        while (i < mid) {
            temp[k++] = prefix[i++];
        }

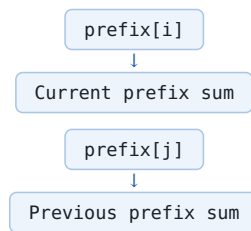
        while (j < right) {
            temp[k++] = prefix[j++];
        }

        for (i = left; i < right; i++) {
            prefix[i] = temp[i - left];
        }

        return count;
    }
}

```

CODE KA MAIN LOGIC



Subarray Sum:

$\text{prefix}[j] - \text{prefix}[i]$

Valid condition:

```

lower <=
prefix[j] - prefix[i]
<= upper

```

Rearrange:

```

prefix[j] - upper
<= prefix[i]
<= prefix[j] - lower

```

Is range mein jitne prefix sums hain, utne valid subarrays hain.

MERGE SORT KA ROLE

Normal Merge Sort:

Array ko sort karta hai.

Yahan Merge Sort:

1. Prefix sums sort karta hai 2. Valid range ke prefix pairs count karta hai 3. Sorted halves merge karta hai

COMPLEXITY

Time: $O(n \log n)$

Space: $O(n)$

WHY $O(n \log n)$?

Merge Sort:

Level 1: $O(n)$

Level 2: $O(n)$

...

Total levels: $O(\log n)$

Therefore:

$O(n \log n)$

QUICK NOTE

Problem: Count of Range Sum

Input: nums, lower, upper

Output: Number of subarrays
Pattern: Prefix Sum + Merge Sort
Core:

```
prefix[i] = prefix[i-1] + nums[i]
```

Subarray Sum:
 $\text{prefix}[j] - \text{prefix}[i]$
Condition:

```
lower <=
prefix[j] - prefix[i]
<= upper
```

Rearranged:

```
prefix[j] - upper
<= prefix[i]
<= prefix[j] - lower
```

Optimization:
Merge Sort + Two Pointers
Time: $O(n \log n)$
Space: $O(n)$

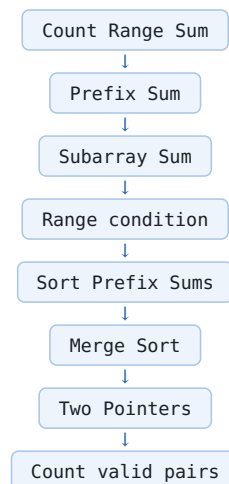
IMPORTANT INTERVIEW POINT

Normal Prefix Sum + HashMap yahan directly enough nahi hai.
Reason:
Humein exact sum nahi chahiye.
Humein RANGE chahiye:

```
lower <= sum <= upper
```

Isliye:
Prefix Sum + Sorted Prefix Sums + Merge Sort + Two Pointers

FINAL PATTERN



Q15 Minimum Window Substring

INPUT

```
String s
String t
```

OUTPUT

```
String

Matlab:
String s ke andar sabse choti window/substring
return karni hai jisme t ke saare characters
required frequency ke saath present hon.
```

EXAMPLE

```
s = "ADOBECODEBANC"
t = "ABC"
```

Output:

"BANC"

WHY?

"BANC" mein:

```
A -> 1
B -> 1
C -> 1

t = "ABC"
```

Teeno characters required frequency ke saath present hain.

Aur "BANC" minimum valid window hai.

ANOTHER EXAMPLE

```
s = "a"
t = "a"
```

Output:

"a"

ANOTHER EXAMPLE

```
s = "a"
t = "aa"
```

Output:

""

Kyunki s mein 2 'a' nahi hain.

IMPORTANT

Character sirf present hona enough nahi hai.

Frequency bhi match honi chahiye.

Example:

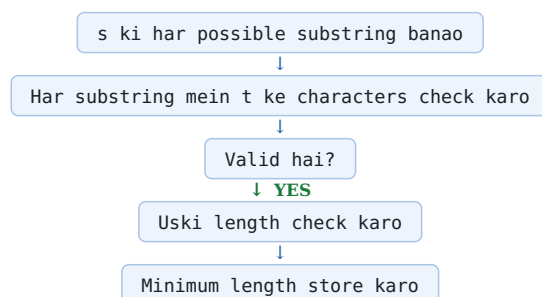
```
s = "AAABBC"
t = "AABC"
```

Required:

```
A -> 2
B -> 1
C -> 1
```

Window mein minimum ye sab hona chahiye.

BRUTE FORCE



BRUTE FORCE

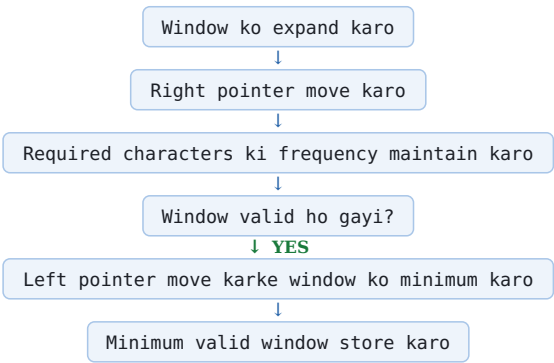
Time: $O(n^2 \times m)$

```
n = length of s
m = length of t
```

OPTIMIZED APPROACH

Sliding Window + HashMap

MAIN IDEA



HASHMAP

t ke characters ki required frequency store karenge.

Example:

```
t = "AABC"

Map:

A -> 2
B -> 1
C -> 1
```

WINDOW

s ke characters ki current frequency maintain karenge.

Example:

Window:

"ADOBEC"

Current:

```
A -> 1
B -> 1
C -> 1
```

Lekin t:

```
A -> 1
B -> 1
C -> 1
```

Window valid hai.

IMPORTANT CONCEPT

```
graph TD
    A[required] --> B[t mein kitne total characters chahiye]
    B --> C[formed]
    C --> D[Current window mein kitne required characters ki frequency satisfy ho gayi]
```

VALID WINDOW

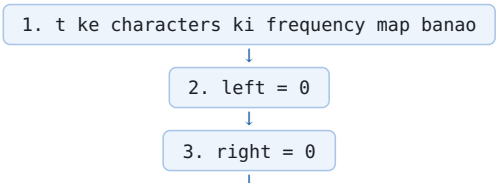
```
formed == required
```

Jab:

```
formed == required
```

window valid hai.

WORKFLOW





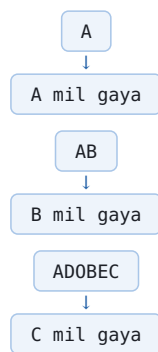
EXAMPLE

```
s = "ADOBECODEBANC"  
t = "ABC"
```

Required:

```
A -> 1  
B -> 1  
C -> 1
```

Window expand:



```
formed == required
```

Window valid:

"ADOBEC"

Ab left move karenge taaki window choti ho.

Eventually:

"BANC"

Minimum valid window:

"BANC"

IMPORTANT CONDITION

Current character required hai ya nahi:

```
if (need.containsKey(ch))
```

Frequency update:

```
window.put(  
    ch,  
    window.getOrDefault(ch, 0) + 1
```



```
)

Character ki requirement satisfy hui:

if (window.get(ch).intValue() == need.get(ch).intValue()) {
    formed++;
}
```

WINDOW SHRINK

```
while (formed == required)
```

Current window valid hai, isliye left ko move karenge aur minimum window find karenge.

MINIMUM WINDOW UPDATE

```
if (right - left + 1 < minLength) {

    minLength = right - left + 1;

    start = left;
}
```

LEFT CHARACTER REMOVE

```
char leftChar = s.charAt(left);

if (need.containsKey(leftChar)) {

    window.put(
        leftChar,
        window.get(leftChar) - 1
    );

    if (window.get(leftChar) < need.get(leftChar)) {
        formed--;
    }
}

left++;
```

COMPLETE CODE

```
import java.util.HashMap;

class Solution {

    public String minWindow(String s, String t) {

        if (s.length() < t.length()) {
            return "";
        }

        HashMap<Character, Integer> need = new HashMap<>();
        HashMap<Character, Integer> window = new HashMap<>();

        // t ki frequency count
        for (int i = 0; i < t.length(); i++) {

            char ch = t.charAt(i);

            need.put(
                ch,
                need.getOrDefault(ch, 0) + 1
            );
        }

        int left = 0;

        int formed = 0;

        int required = need.size();

        int minLength = Integer.MAX_VALUE;

        int start = 0;

        // Sliding Window
        for (int right = 0; right < s.length(); right++) {

            char ch = s.charAt(right);

            // Current window mein character add karo
            window.put(
                ch,
                window.getOrDefault(ch, 0) + 1
            );
```

```

// Required frequency satisfy hai?
if (need.containsKey(ch)
    && window.get(ch).intValue()
    == need.get(ch).intValue()) {

    formed++;
}

// Window valid hai
while (formed == required) {

    // Minimum window update
    if (right - left + 1 < minLength) {

        minLength = right - left + 1;

        start = left;
    }

    // Left character remove karo
    char leftChar = s.charAt(left);

    if (need.containsKey(leftChar)) {

        window.put(
            leftChar,
            window.get(leftChar) - 1
        );

        if (window.get(leftChar)
            < need.get(leftChar)) {

            formed--;
        }
    }

    left++;
}

if (minLength == Integer.MAX_VALUE) {
    return "";
}

return s.substring(
    start,
    start + minLength
);
}
}

```

COMPLEXITY

Let:

```

n = length of s
m = length of t

```

Time: $O(n + m)$

Space: $O(n + m)$

Usually character set fixed ho to space effectively $O(1)$ bhi maana ja sakta hai.

QUICK NOTE

Problem: Minimum Window Substring

Pattern: Sliding Window + HashMap

Input: s, t

Output: Minimum substring of s containing all characters of t.

```

HashMap 1:
need

Character -> required frequency

HashMap 2:
window

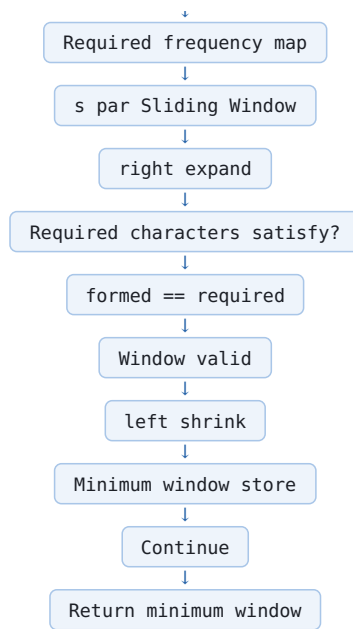
Character -> current window frequency

```

Important variables:

left right formed required minLength start

CORE LOGIC



MOST IMPORTANT

Expand:

right++

Shrink:

```
while (formed == required)
    left++
```

Valid:

```
formed == required
```

Minimum:

right - left + 1

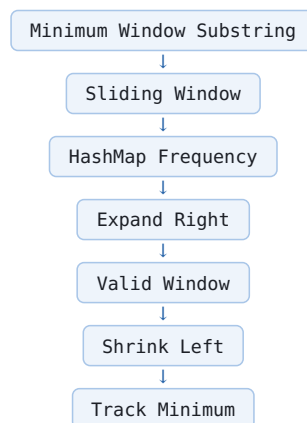
EXAMPLE REVISION

```
s = "ADOBECODEBANC"
t = "ABC"
```

Final answer:

"BANC"

INTERVIEW PATTERN



COMMON MISTAKES

1. Duplicate characters ki frequency ignore karna.

Wrong:

```
t = "AABC"
```

Sirf A, B, C check karna.

Correct:

```
A -> 2
B -> 1
C -> 1
```

2. Window valid hote hi stop kar dena.

Wrong.

Valid window milne ke baad left ko shrink karna zaroori hai.

3. formed ko har character ke liye increase kar dena.

Sirf tab increase hoga jab:

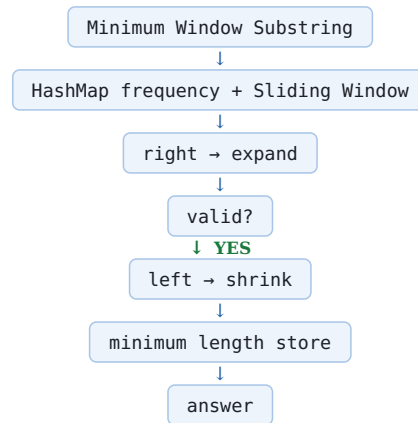
```
window frequency == required frequency
```

4. Left character remove karte waqt formed incorrectly decrease karna.

Sirf tab decrease hoga jab:

window frequency < required frequency.

FINAL REVISION



Q15 COMPLETE